

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

ОТЧЁТ

Лабораторная работа №3
Одностраничное приложение на Vue

Выполнил:
студент группы J3212
Мирасов Константин

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Реализовать AxonHub как одностраничное приложение на Vue. Организовать маршрутизацию, загрузку данных через API, авторизацию и пользовательские действия. Разделить интерфейс на компоненты, а повторно используемую реактивную логику — на composables.

Ход работы

1. Архитектура приложения

web/src/views содержит страницы маршрутов, components — элементы интерфейса, services — HTTP-запросы и преобразование данных. В composables находятся загрузка, сессия, тема и социальные действия. App выводит страницу через RouterView; PageLayout объединяет шапку, основное содержимое и подвал.

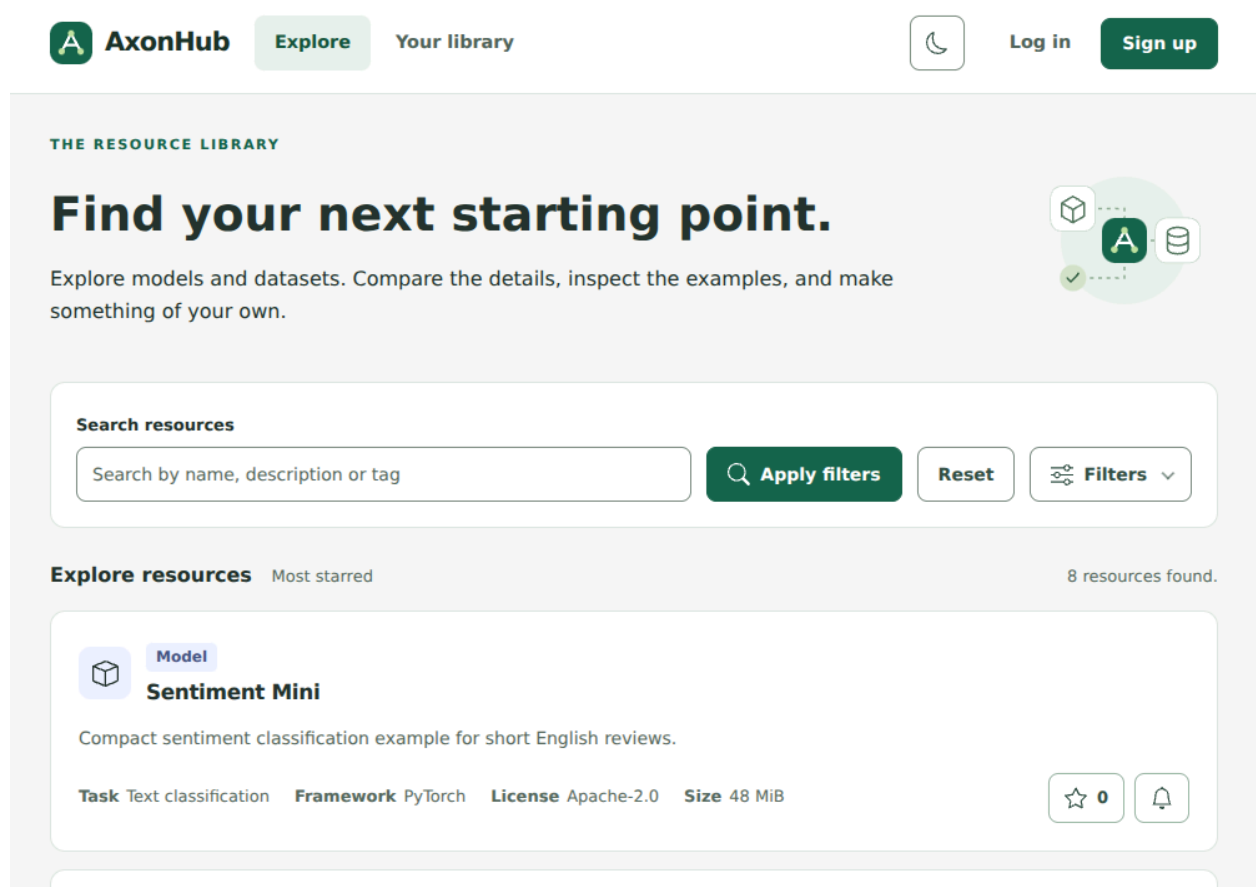


Рисунок 1. Каталог одностраничного приложения

Vue отвечает за реактивное представление, Vue Router — за навигацию, Axios — за HTTP API. Vite передаёт запросы /api локальному серверу на порту 3001.

2. Маршруты и доступ к кабинету

Используется hash history: адрес содержит путь после знака решётки, поэтому прямое открытие и перезагрузка не требуют серверной обработки маршрутов приложения.

Маршрут	Содержимое
/explore	Каталог и фильтры
/resources/:id	Описание ресурса и действия
/login, /register	Вход и создание аккаунта
/profile	Ресурсы и подписки пользователя
Неизвестный путь	Страница Page not found

Кабинет помечен `requiresAuth`. Guard перенаправляет гостя на вход и сохраняет целевой адрес в `returnTo`.

web/src/router/index.js · строки 51-55

```
router.beforeEach((to) => {  
  if (to.meta.requiresAuth && !session.value) {  
    return { path: '/login', query: { returnTo: to.fullPath } };  
  }  
});
```

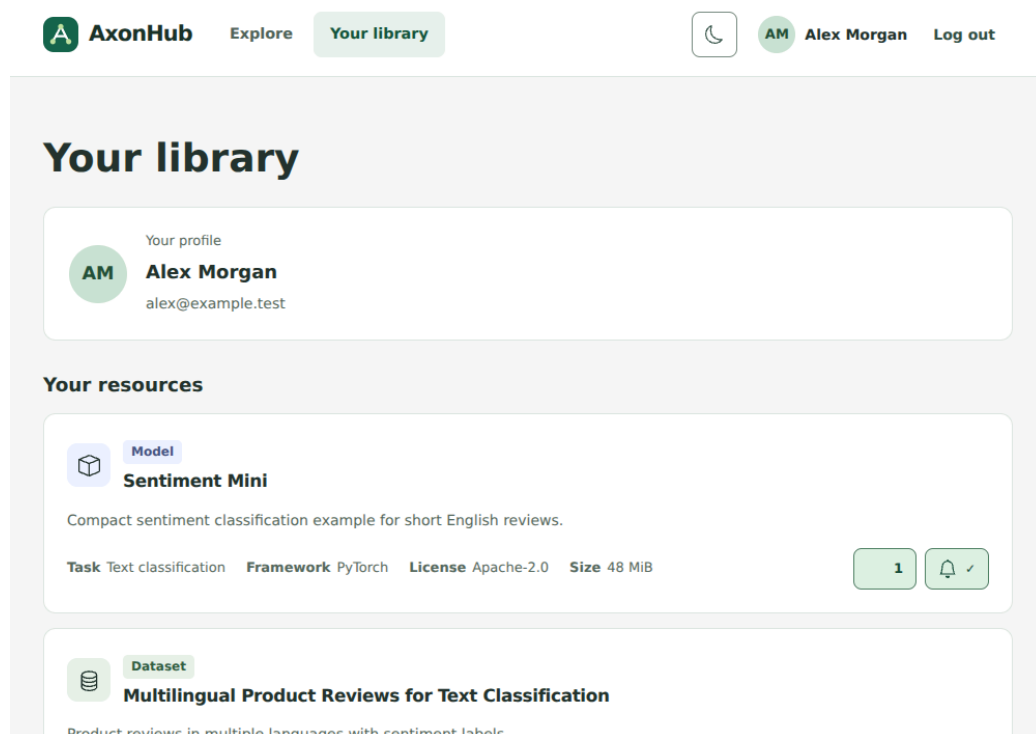


Рисунок 2. Кабинет авторизованного пользователя

3. HTTP-сервисы и загрузка

Axios-клиент задаёт общий адрес API и тайм-аут. `requestJson` передаёт параметры запроса и при необходимости токен. Сервис ресурса получает данные, а компонент использует результат без деталей HTTP.

web/src/services/http.js · строки 10–15

```
const response = await api.request({
  url: path,
  method,
  data: body,
  headers: authenticated && token ? { Authorization: `Bearer ${token}` } : {},
});
```

`useLoad` возвращает `data`, `loading`, `error` и `reload`. Номер запроса не позволяет запоздалому ответу заменить более новый результат; при уничтожении компонента текущая версия становится неактуальной.

web/src/composables/useLoad.js · строки 15–19

```
const result = await loader();

if (request === version) {
  data.value = result;
}
```

При сетевой ошибке пользователь получает сообщение и кнопку повторения. Состояние загрузки представлено явно, повторные действия на время запроса блокируются.

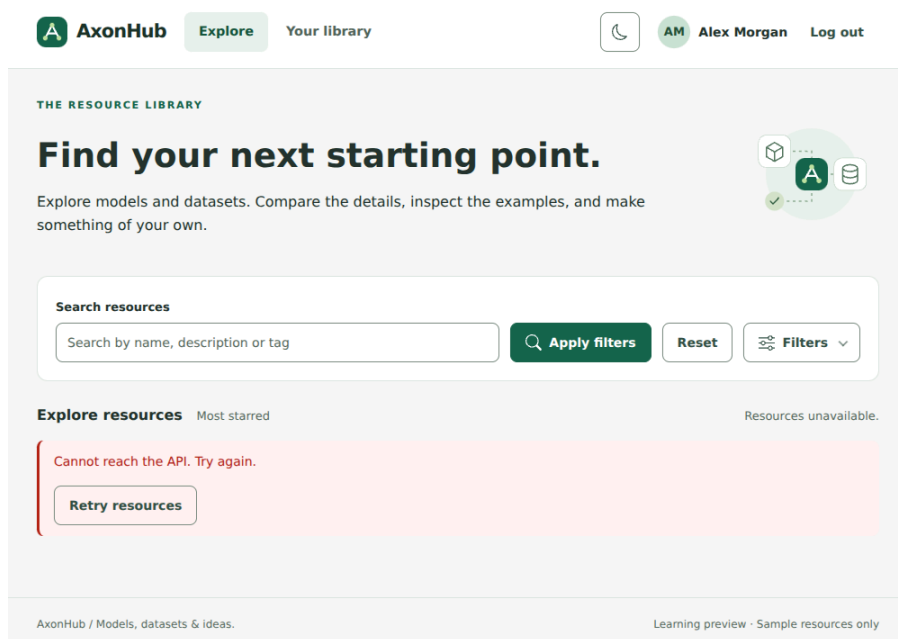


Рисунок 3. Сообщение об ошибке API и повторная загрузка

4. Компоненты страницы ресурса

ResourceView собирает сведения о ресурсе, документацию, действия и обсуждение. Отдельные компоненты ResourceInformation, ResourceDocumentation и ResourceFork отвечают за свои части страницы. ResourceRow переиспользуется для строк каталога и библиотеки.

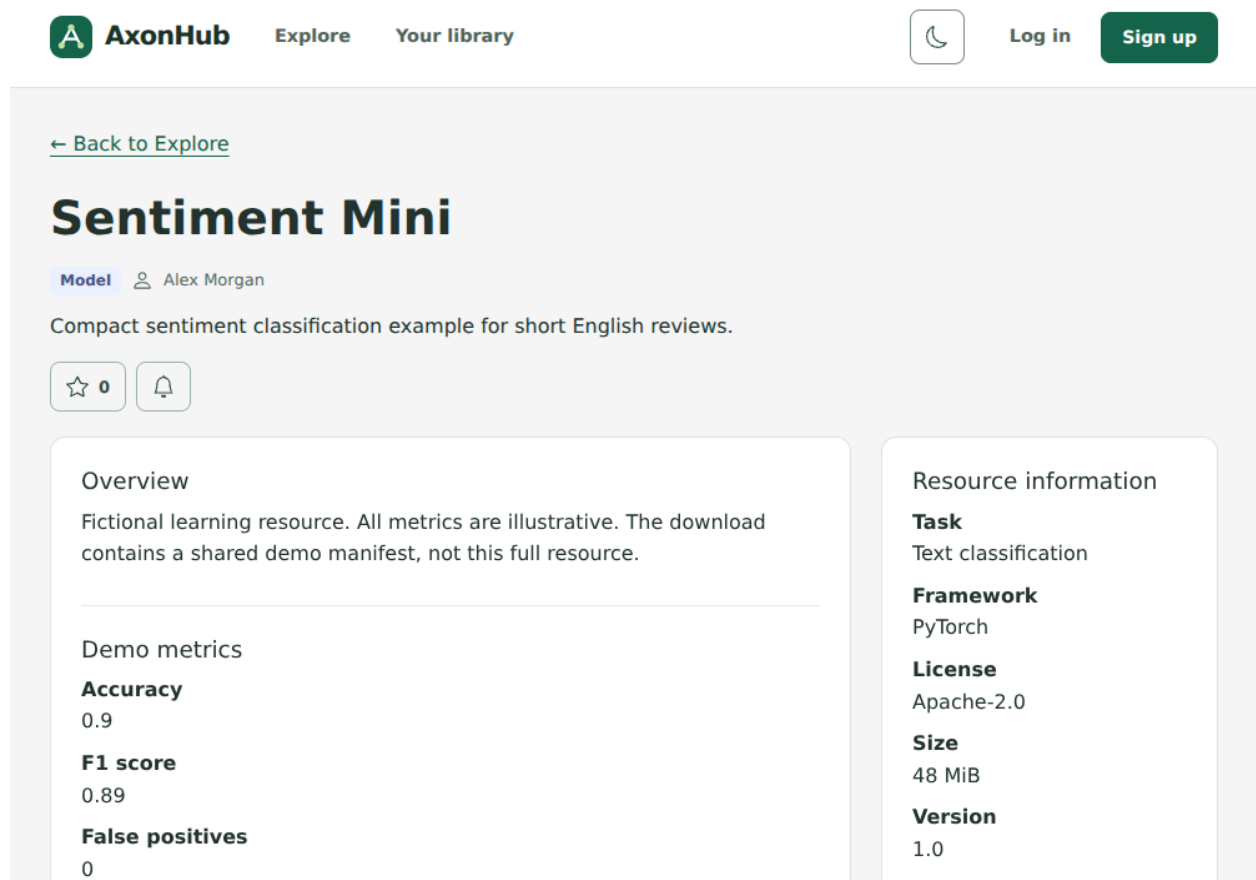


Рисунок 4. Страница модели в Vue-приложении

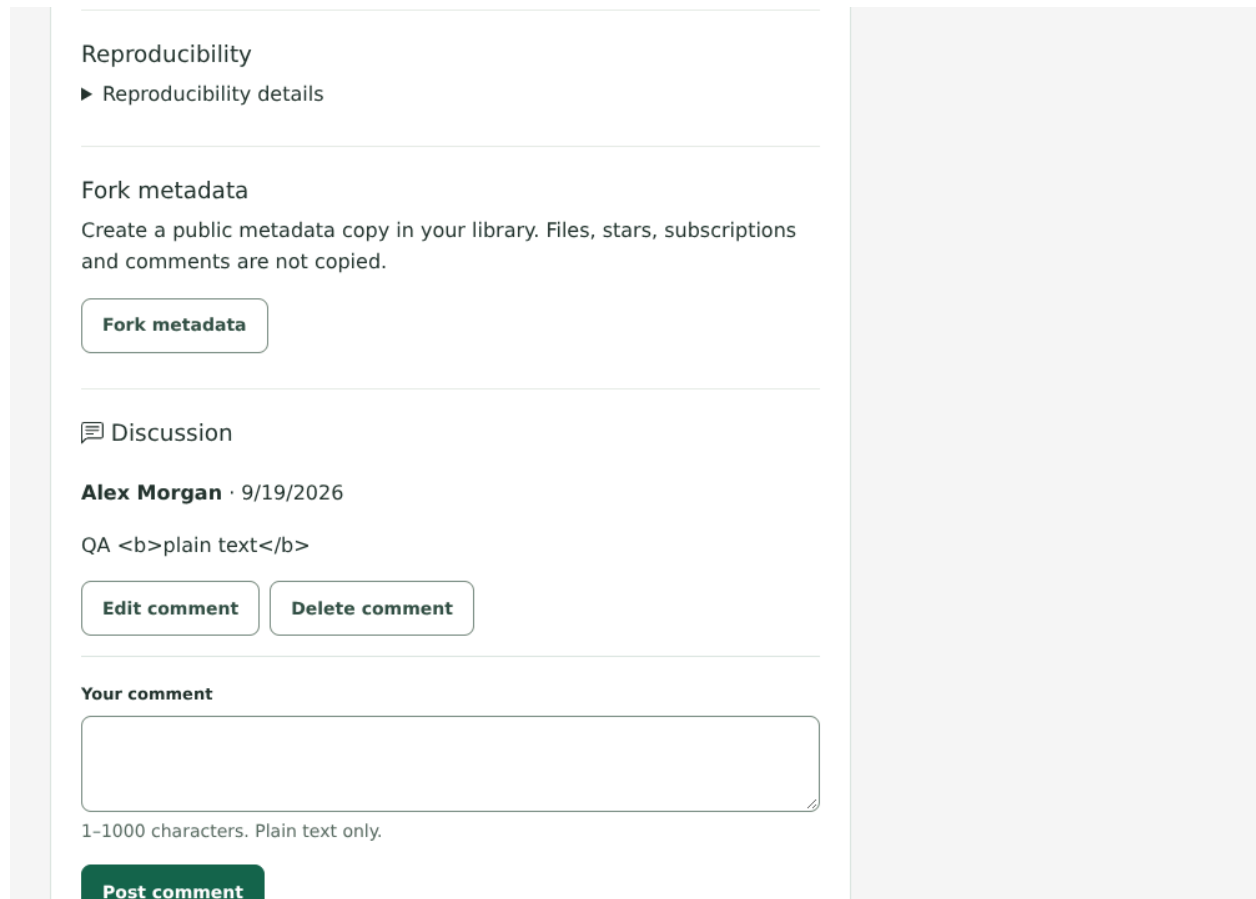
ResourceFilters передаёт выбранные параметры событиями. Значения фильтров отражаются в URL, поэтому адрес можно открыть повторно с тем же набором условий. ResourceActions показывает звезду, подписку, их состояние и сообщения операции.

Общее состояние сессии находится в useSession. useResourceActions согласует звёзды и подписки между строкой каталога и страницей одного ресурса. useTheme управляет цветовой схемой и её сохранением.

5. Сессия и обсуждение

AuthForm объединяет вход и регистрацию. Ошибки привязаны к полям, а успешный ответ сохраняет безопасные данные сессии. Выход очищает сессию; недействительный токен вызывает уведомление о необходимости повторного входа.

DiscussionForm сохраняет черновик при ошибке запроса. DiscussionList выводит комментарии и действия владельца. Подтверждение удаления отделено от первого нажатия; после завершения операции фокус возвращается к подходящему элементу.



The screenshot displays a web interface for a discussion. It features a sidebar on the left with a light gray background. The main content area has a white background and is divided into sections by horizontal lines. The first section is titled 'Reproducibility' and contains a link '► Reproducibility details'. The second section is titled 'Fork metadata' and includes the text 'Create a public metadata copy in your library. Files, stars, subscriptions and comments are not copied.' Below this text is a button labeled 'Fork metadata'. The third section is titled 'Discussion' and shows a comment by 'Alex Morgan' dated '9/19/2026'. The comment text is 'QA plain text'. Below the comment are two buttons: 'Edit comment' and 'Delete comment'. The fourth section is titled 'Your comment' and contains a large text input field. Below the input field is the text '1-1000 characters. Plain text only.' and a green button labeled 'Post comment'.

Рисунок 5. Обсуждение ресурса с обычным текстом комментария

Уведомление об успешном действии закрывается через две секунды. Сообщение об ошибке остаётся до закрытия пользователем; автоматическое исчезновение уведомления не перехватывает фокус ввода.

6. Сохранение действий и копирование ресурса

Звёзды, подписки и комментарии хранятся в API. Кнопки отражают состояние операции, а повторное нажатие во время запроса блокируется. Подписанные ресурсы доступны в личной библиотеке.

Fork metadata создаёт новую запись с копией описания, текущим владельцем и ссылкой на оригинал. Повторное действие открывает уже существующую копию пользователя. Download предоставляет демонстрационный манифест, не файлы обучения модели.

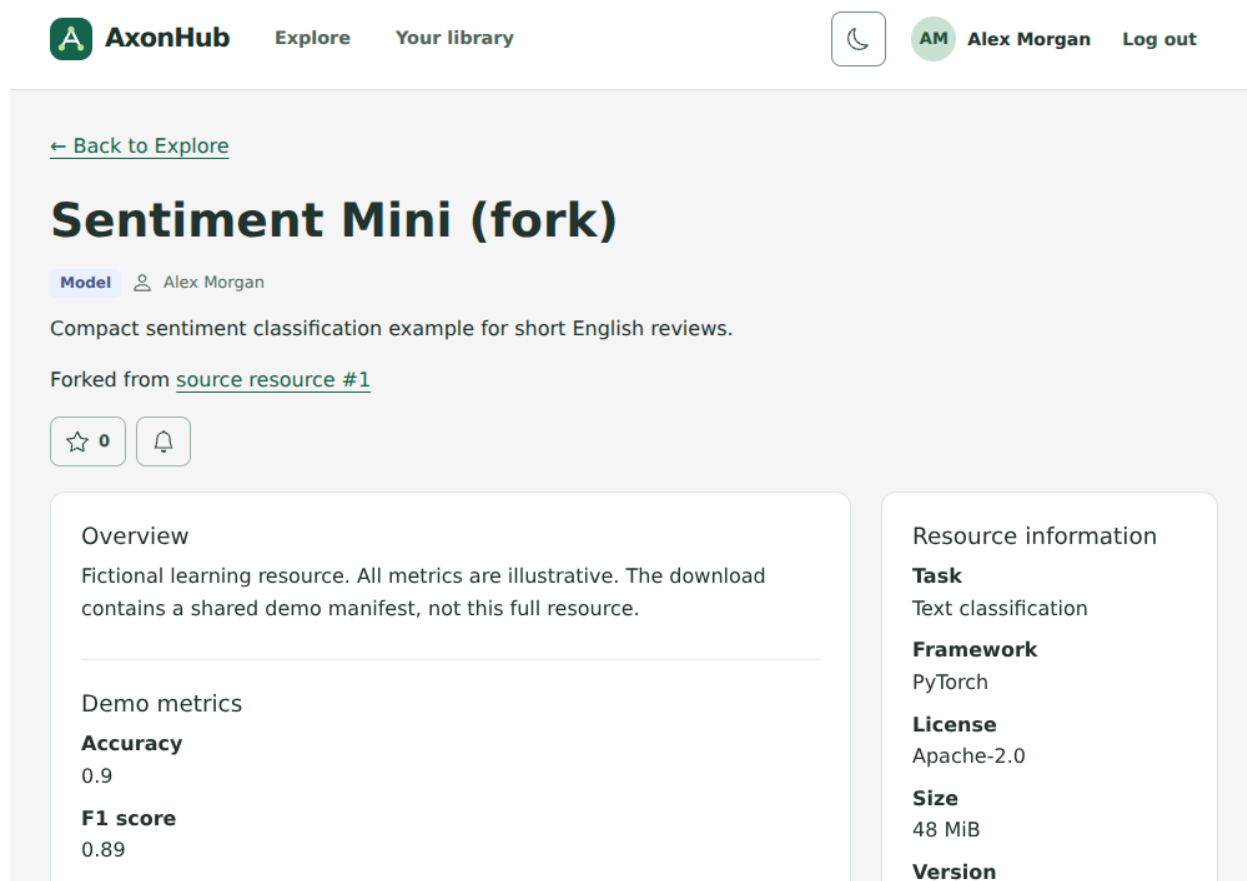


Рисунок 6. Созданная копия метаданных ресурса

Вывод

AxonHub реализован как SPA с маршрутизацией, Axios-сервисами, авторизацией и сохранением пользовательских действий. Компоненты отделяют представление от запросов, composables объединяют повторно используемую логику. Сохранены адаптивность, доступные элементы интерфейса, темы и общий SVG-спрайт.